

Full length article

Fine-tuning large language models with contrastive margin ranking loss for selective entity matching in product data integration

Qian Ruan^a, Dachuan Shi^{b,*}, Thomas Bauernhansl^c^a Technical University of Darmstadt, Department of Computer Science, Hochschulstraße 10, 64289 Darmstadt, Germany^b Fraunhofer Institute for Manufacturing Engineering and Automation IPA, Nobelstraße 12, 70569 Stuttgart, Germany^c Universität Stuttgart, Institute of Industrial Manufacturing and Management IFF, Allmandring 35, 70569 Stuttgart, Germany

ARTICLE INFO

Keywords:

Entity matching

Entity Resolution

Large language model

Contrastive learning

Data integration

ABSTRACT

Entity Matching (EM) concerns identifying entities from different data sources that correspond to the same real-world object. It is widely used for product data integration in e-commerce, product classification, and inventory management, enabling the matching of duplicate product records with heterogeneous descriptions across various platforms and software systems. The standard EM solution consists of two steps: a blocking step to retrieve a subset of candidates and a pairwise matching step to classify whether the query entity matches each candidate. However, a significant challenge arises when pairwise matching fails to account for similar distractors within the candidate subset, often leading to false positive matches. This issue has been largely overlooked in prior work and existing benchmark datasets. In this study, we address this gap through three key aspects. First, we revisit the standard pairwise EM setting by recompiling existing benchmark datasets to include more hard negative (HN) candidates, which are semantically similar to corresponding query entities. We then evaluate state-of-the-art (SOTA) pairwise matchers on these recompiled datasets, revealing the limitations of the conventional pairwise EM approach under more challenging and realistic conditions. Second, we propose a selective EM approach that formulates EM as a listwise selection task, where the query entity is compared directly with the entire candidate set rather than evaluated through independent pairwise classifications. Accordingly, a new evaluation framework is introduced, including recompiled benchmark datasets and a new evaluation metric. Third, we propose a selective EM method Mistral4SelectEM, which fine-tunes a large language model for selective EM by structuring it into a Siamese network and fine-tuning it with a novel contrastive margin ranking loss (CMRL). It aims to enhance the model's ability to distinguish true positives from semantically similar HNs. Extensive experiments demonstrate that our method outperforms SOTA pairwise EM approaches in both efficiency and performance across multiple benchmark datasets. The code and the recompiled entity matching benchmark datasets are publicly available at: <https://github.com/quickhdsdc/LLM4EntityMatching>.

1. Introduction

Entity Matching (EM), often called entity resolution, has been widely applied for product data integration and record deduplication across various application domains [1]. In e-commerce, EM plays a critical role in identifying and merging duplicate product listings in databases across different sellers or platforms [2]. Within the supply chain, EM supports inventory management by detecting duplicate or mismatched product material records across warehouses or systems [3]. In manufacturing, EM facilitates the integration of information related to products, processes, and resources from heterogeneous sources, enabling the

formation of comprehensive knowledge representations [4,5].

The EM task aims to identify entities in different data sources that represent the same real-world entity [6]. Given a query entity, EM commonly involves a blocking method and a matcher. The blocker selects a subset of the most likely matching candidates from the target dataset. Afterward, the matcher compares the similarity of the query entity and a candidate in a pairwise manner to decide whether they are matched. This pairwise comparison has to be repeated for each candidate selected by the blocker. This setting has become the standard task formulation both in some early work [6] and the neural-network-based approaches [7].

* Corresponding author.

E-mail address: dachuan.shi@ipa.fraunhofer.de (D. Shi).<https://doi.org/10.1016/j.aei.2025.103538>

Received 30 January 2025; Received in revised form 15 May 2025; Accepted 30 May 2025

Available online 9 June 2025

1474-0346/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

However, standard pairwise EM approaches compare the query and candidate entity independently. Since blockers typically rely on similarity-based retrieval [7], the resulting candidate subset contains semantically similar candidates. This can make it challenging for pairwise EM to distinguish between the true match and its similar distractors. Consequently, it frequently results in false positive matches [8]. Fig. 1 illustrates this issue on the right side. In this example, the query entity describing a software product originates from the Amazon e-commerce database. The true positive match is Candidate Entity 3. In pairwise EM, a binary label (match or not) is predicted for each pair of query and candidate entities. However, contextual information, such as the presence of competing candidates, is not included in the input. This results in a false positive match for Candidate Entity 1, which is semantically similar to the query. Additionally, the pairwise comparison must be performed for each candidate individually. This leads to increased latency in responding to the query.

The left side of Fig. 1 shows the concept of the proposed selective EM approach. Instead of evaluating candidate pairs independently, the entire candidate subset is processed in a single forward pass. For each query, the model performs contrastive learning over the full candidate list. This enhances the model's ability to distinguish true positives from similar distractors. The prediction is the index of the most matching candidate.

It is worth noting that the end-to-end EM process aligns closely with information retrieval (IR). In particular, pairwise EM resembles pointwise reranking in IR, whereas our proposed selective EM approach is similar to listwise reranking. However, despite this structural similarity, the underlying task objectives differ significantly. Listwise reranking in IR aims to rank passages by their graded relevance to a query. In contrast, selective EM is designed to identify the true positive match among a candidate set, with no interest in the relative ranking of negative candidates. While our formulation shares the listwise structure of learning-to-rank models, it redefines the objective: instead of producing a relevance-ordered list, the model focuses on discriminating the true match from hard negatives. This distinction is critical, as it changes both the evaluation focus and the learning target. A detailed review and discussion of recent listwise reranking methods is provided in Section 2.2.

Furthermore, the evaluation of all the existing EM methods relied on standard benchmark datasets [9] and the F1 score as the performance metric. These datasets, however, do not explicitly include multiple semantically similar candidates for each query entity. This limits their ability to reveal how well EM methods can handle hard negative (HN)

candidates. Fig. 2 illustrates the sample distribution in the test set of the Amazon-Google benchmark dataset, which is a widely used EM benchmark dataset. This test set contains 2293 samples, each consisting of a query-candidate entity pair. The chart on the right shows that $\sim 10.2\%$ of test samples are positive pairs, while the majority ($\sim 71.3\%$) are non-competitive negatives. Using the query entities in labeled positive samples as anchors, non-competitive negatives (CNs) refer to candidate entities that are semantically irrelevant to any query entity, making them easy to classify. In contrast, CNs are semantically relevant to a query entity and thus more difficult to distinguish. CNs account for $\sim 18.4\%$ of the test set. HNs form a subset of CNs and are the most challenging to classify correctly. The histogram on the left highlights the distribution of CNs per query entity. Most queries have only a few CN, with five being the most common (accounting for 20 % of CNs). This means that only around 17 query entities (calculated as $2293 \times 18.4\% \times 20\% \div 5 \approx 17$) in the test set contain one positive sample and five CNs. Even among these, the actual number of HNs per query is often less than five. Such a dataset composition is insufficient to evaluate whether an EM method can effectively distinguish true positives from semantically similar distractors.

In this study, we address the identified gap through three key aspects. First, we revisit and empirically examine the standard pairwise EM approach. To do so, we recompile benchmark datasets to ensure that each query entity has at least one true positive and several semantically similar HNs. This setup closely reflects real-world scenarios, where blockers often retrieve such competitive candidates and thus provide a more challenging evaluation setting. By comparing performance metrics, such as F1 score, precision, and false positive rate, between the original and recompiled datasets, we reveal critical limitations of state-of-the-art (SOTA) pairwise matchers.

Second, we propose a selective EM approach that frames the task as a listwise selection problem. Instead of comparing each query-candidate pair independently, the selective EM considers the entire candidate set in a single step. This setup aligns with how EM systems are used in practice. Typically, systems generate a ranked list of likely matches and rely on human experts to make the final decision. In this process, predicted positives are prioritized to help users quickly locate the best matches. To evaluate this selective EM setting, we introduce a new evaluation framework, including our recompiled datasets and a new metric: Mean Reciprocal Rank with Penalty (MRRP). MRRP extends the MRR metric widely used in IR by incorporating penalties for false positives and missed detections. It captures two practical requirements of EM: improving the ranking of true positives and minimizing false

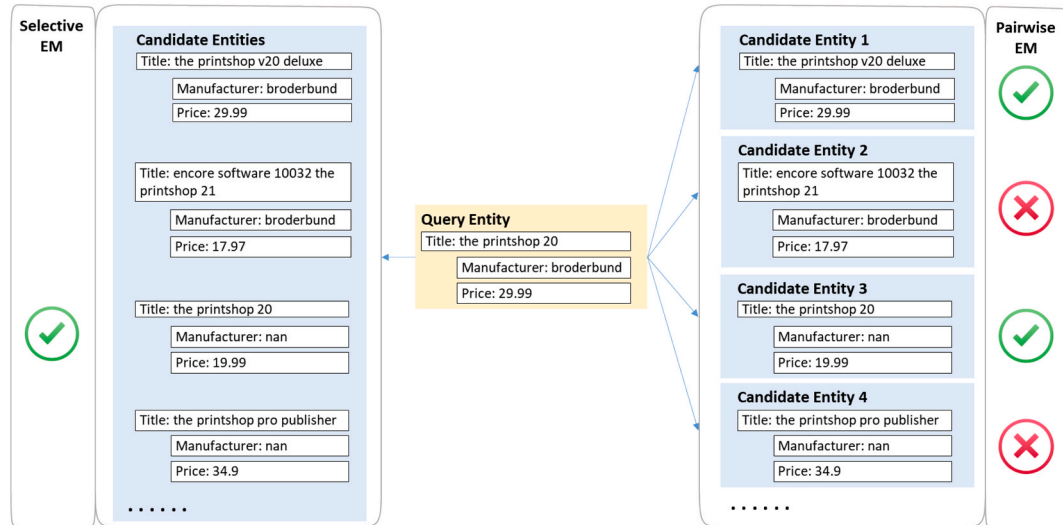


Fig. 1. Comparison of pairwise and selective EM using exemplary entities from the benchmark dataset Amazon-Google [9] that describes software products. The query entity originates from the Amazon database, while the candidate entities are sourced from the Google database.

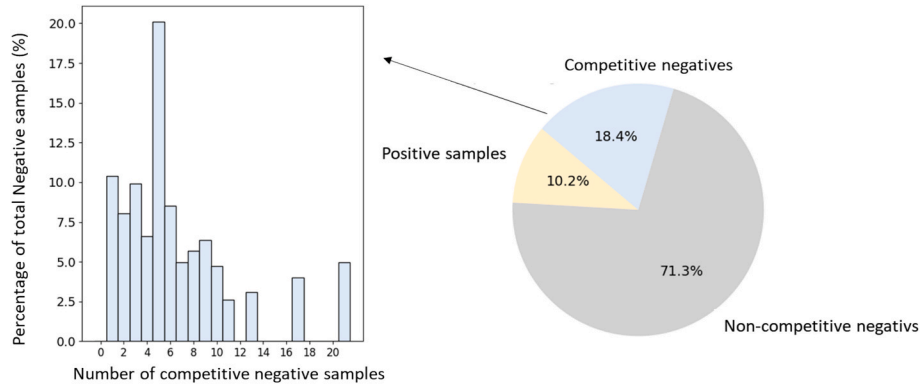


Fig. 2. Sample distribution in the Amazon-Google test set. CNs are semantically relevant to a query entity in the labeled positive samples. *Non-competitive negatives* are semantically irrelevant to any query entities. The right chart shows the overall distribution; the left histogram shows the number of CNs per query.

classifications. This also highlights a key distinction between selective EM and IR. In IR, metrics such as Normalized Discounted Cumulative Gain (NDCG) are widely used to assess the quality of ranked relevance lists. In contrast, selective EM is concerned only with identifying and promoting the true match. The ranking of negative candidates is irrelevant, making NDCG unsuitable for this task.

Third, we propose a method to fine-tune large language models (LLMs) for selective EM, termed *Mistral4SelectEM*, by structuring the embedding model used for blocking into a Siamese network and fine-tuning it with a novel contrastive margin ranking loss (CMRL). Unlike existing LLM-based entity matchers that rely on prompt engineering [8,13,14] or generative fine-tuning [5,15], *Mistral4SelectEM* leverages a pre-trained embedding model, which is typically trained on large datasets in diverse tasks, as the backbone of the Siamese network for separately encoding query and candidate entities. Softmax-normalized similarity scores are computed across all candidates for a given query, representing the likelihood of each being the true match. During training, CMRL encourages positive candidates to rank higher by penalizing negative candidates with similar scores. To further enhance ranking robustness, a margin is introduced as a safety buffer to increase the separation between positive and negative candidates. This enforces the desired relative ordering of scores, with positive candidates consistently ranked above negative ones. Our key contributions are summarized as follows:

- We revisit the standard pairwise EM setting by recompiling existing benchmark datasets and conducting comprehensive evaluations on SOTA pairwise EM methods. The experimental findings reveal the inherent limitations of the standard pairwise EM approach, particularly its vulnerability to false positives when handling semantically similar candidates.
- A novel selective EM approach is proposed that reformulates EM as a listwise selection task, enabling the model to compare a query entity against the entire candidate subset in a single forward pass. To support this paradigm, we introduce a new evaluation framework comprising recompiled benchmark datasets and MRRP as the evaluation metric. This new setting better reflects real-world EM scenarios by assessing a model's ability to refine blocking results.
- To implement selective EM, we propose and validate the method named *Mistral4SelectEM* that leverages an LLM-based embedding model structured in a Siamese network and fine-tuned with the proposed CMRL. *Mistral4SelectEM* enhances the model's ability to distinguish true positives from semantically similar negatives, thereby improving matching accuracy and consistency.

The remainder of this paper is organized as follows: [Section 2](#) reviews SOTA EM methods and related IR approaches. [Section 3](#) details the selective EM approach and the evaluation framework. [Section 4](#)

evaluates existing pairwise EM methods and validates the proposed selective EM approach, respectively. Finally, [Section 5](#) presents the conclusions.

2. Related work

2.1. Entity matching

EM often deals with structured data such as databases, where information is organized in tables with rows representing records and columns representing attributes [6]. In this context, entity matching focuses on identifying records (rows) that correspond to the same real-world entity by comparing their attributes (columns). EM involves multiple steps, including schema alignment, blocking, and entity pair comparison [7]. Schema matching aligns related attributes between datasets to enable effective pairwise comparisons. However, recent EM approaches often bypass explicit schema alignment by concatenating all or selected attributes into a single input string for language models [10–16]. This strategy helps mitigate issues arising from heterogeneous schemas and noisy or incomplete data [17]. Schema alignment is a primary concern in the field of entity alignment, which specifically addresses knowledge graphs and ontologies [18]. While entity alignment and EM share similar techniques, such as attribute embedding and representation learning, they differ significantly in their task formulations, benchmark datasets, and evaluation metrics [18–20]. In the following, we focus mainly on the EM task.

Blocking reduces computational costs by retrieving a subset of relevant candidates. Traditional methods like hashing have given way to the pre-trained language model (PLM) based semantic search [21]. For each query entity, the top- K most similar candidate entities are retrieved using a PLM-based embedding model combined with indexing and approximate nearest neighbor search methods. This technique is akin to the initial retrieval step in the IR task.

The core of the EM task lies in entity pair comparison. The evolution of EM methods showcases a transition from statistical feature learning [6] to training deep learning models [17,20], to fine-tuning PLMs [10–12]. DeepMatcher [9] explored a design space of deep learning models for EM, ranging from simple pooling-based embeddings to recurrent neural networks and attention-based architectures. These models operated on aligned attributes. In contrast, Seq2SeqMatcher [17] concatenated attributes into a sequence and performed EM as a sequence classification task using attention mechanisms and convolutional neural networks. This design avoids the need for explicit schema alignment and has become one of the most common preprocessing strategies in modern EM approaches. Subsequent research leveraged PLMs for EM and has achieved SOTA results. Ditto [10], as a representative PLM-based approach, fine-tuned models such as BERT by incorporating optimization measures like summarization of long attribute

strings and augmentation with challenging training examples. Similarly, several PLMs were compared [11], showing that fine-tuned RoBERTa achieved the best performance for EM. JointMatcher [12] further enhanced EM by manipulating the learned intermediate embeddings.

Recently, LLMs have demonstrated remarkable capabilities in semantic understanding and complex problem-solving. Unlike traditional PLMs, LLMs enable users to interact directly through natural language prompts without fine-tuning the models' weights. Peeters and Bizer [13] explored prompting techniques like zero-shot prompting (ZSP) and in-context learning (ICL) for EM tasks. A powerful LLM, such as GPT-4, using ICL can achieve performance comparable to fine-tuned PLMs [13]. APrompt4EM [14] explored prompt tuning, where soft tokens were introduced to adapt the model for EM without modifying its weights. Beyond prompting, fine-tuning LLMs for EM has been shown to further improve match performance. Steiner et al. [15] fine-tuned LLMs such as LLaMA3-8B and GPT-4o-m to generate binary responses (e.g., yes/no) for given entity pairs. Our previous work [5] demonstrated that a medium-sized LLM, such as LLaMA2-13B, can be fine-tuned using the low-rank adaptation (LoRA) technique with carefully optimized hyperparameters, including LoRA rank and batch size, to outperform both fine-tuned PLMs and LLMs with ICL.

The aforementioned related works are limited to the standard pairwise EM setting. A recent study [8] identified the shortcomings of pairwise EM and empirically evaluated two other EM strategies using LLMs with ZSP: selective EM and pair-comparison EM. (The pair-comparison method compares two candidate matches for a given query entity.) Among these, selective EM demonstrated superior performance over the other two approaches. To further boost accuracy, the prompting strategies of all three methods were combined into a unified framework called COMEM [8]. However, COMEM did not surpass SOTA results delivered by pairwise matchers [5]. This performance gap is likely attributed to the limitations of employed LLMs (3–11B parameters) with ZSP, rather than inherent weaknesses in the selective EM approach.

2.2. Reranking in information retrieval

IR generally involves two main steps: initial retrieval and reranking. Initial retrieval, akin to blocking in EM, aims to efficiently retrieve a subset of relevant candidates. Reranking refines this subset by assigning relevance scores and reordering candidates to optimize ranking quality [22]. Since reranking is closely related to our proposed selective EM, we review the relevant techniques below.

The learning-to-rank theory categorizes reranking models into pointwise, pairwise, and listwise models. Pointwise reranking evaluates candidates independently, predicting each candidate's relevance score or label in isolation. Technically, these methods resemble pairwise EM. Before the emergence of LLMs, a common approach involved fine-tuning PLMs like BERT as cross-encoders [23]. These cross-encoder models concatenated the query and candidate text, employing a dense layer to predict the relevance score based on the embedding of the special token ([CLS]), typically trained using binary cross-entropy loss [22]. The rise of decoder-only LLMs like LLaMA and Mistral has led to novel reranking approaches, leveraging their generative capabilities and prompting mechanisms. For instance, RankLLaMA [24] was fine-tuned using contrastive loss, leveraging the last token embedding to predict similarity scores. LLMRank [25] leveraged prompting techniques to generate textual relevance labels without fine-tuning the underlying model parameters.

Pairwise reranking compares two candidates at a time to determine their relative order, aligning with the comparewise EM approach proposed in [8]. A notable example is DuoBERT [23], which predicts the relative order by using the special token embedding of the concatenated query and two candidates. For LLMs, pairwise ranking prompting has been introduced to rank documents by performing pairwise comparisons and aggregating win ratios to establish a final order [26].

Pointwise and pairwise reranking approaches cannot consider the entire candidate set simultaneously, often leading to inconsistencies in global rankings. Additionally, their computational complexity increases significantly as comparisons must be repeated for each candidate. In contrast, listwise reranking optimizes the entire ranked list during training. However, listwise methods face notable challenges. Including lengthy contexts can complicate a model's ability to generate meaningful results, and the performance of listwise rerankers is often highly sensitive to the order of candidates in the input [27]. Recent research has explored innovative methods to utilize LLMs for listwise reranking, either through prompting or fine-tuning, to overcome these challenges [28–30]. RankGPT [28] and RankVicuna [29] leverages the language generation capability of LLMs to directly generate a permutation of candidate identifiers in descending order of relevance, e.g. [2] > [3] > [1]. In-context reranking [30] aggregates attention weights across all heads and layers in the LLM to compute token-level ranking scores. This method uses the model's internal representation to infer relevance without fine-tuning. FIRST [31] hypothesizes that LLMs approximate a full ranked list during the generation of the top-ranked candidate. It only uses the output logits of identifier tokens to rank candidates based on decreasing logit values. FIRST fine-tunes LLMs with a joint loss function combining a weighted RankNet loss and the standard language modeling loss. ListT5 [32] modifies the T5 model to accept a query, five candidate passages, and their indices as input, generating the top-1 or top-2 passages for reranking. PE-Rank [33] addresses the context length limitation by transforming passage embeddings instead of processing full-text passages. It employs a two-layer MLP, which is fine-tuned jointly with the LLM to transform the passage embeddings generated during retrieval and concatenate them with query and instruction embeddings.

2.3. Research gaps

From the above review, it is evident that a key distinction between listwise reranking in IR and selective EM lies in the nature of the input and the learning objective. In IR, the primary challenge often stems from the length and complexity of candidate passages, which has driven research toward optimizing models for processing long texts. By contrast, in EM tasks, the candidate entity strings are typically much shorter and structured, reducing the relevance of such optimization strategies. Furthermore, many learning strategies used in listwise reranking, such as those aimed at fine-grained relevance scoring across long ranked lists, are not directly applicable to selective EM. This is because the learning objective differs fundamentally: selective EM focuses on identifying the true match rather than ranking all candidates by graded relevance. As a result, selective EM requires task-specific optimization strategies that emphasize discrimination between true and false matches. This adaptation of listwise structures to entity matching tasks has not been systematically explored in prior EM research. Our work initiates this direction by proposing a selective EM framework that leverages listwise structure but redefines its learning objective and evaluation criteria to better suit the EM context.

In the EM domain, existing work remains confined to the standard pairwise EM setting, where entity pairs are evaluated independently. This approach does not take into account the advancements in modern blocking methods, which can already retrieve semantically similar candidates. Consequently, the emphasis of EM should shift from independently identifying matches to refining these retrieved results, ensuring true positives are distinguished from their similar distractors.

Recognizing this research gap has inspired us to propose the selective EM framework, including the task setting, benchmark datasets, and a tailored evaluation metric. Although it adapts listwise reranking principles specifically for the EM task, the objective of the selective EM differs from the listwise reranking. The specific order among positive candidates or negative ones is not crucial. Instead, the primary goal is to identify the positive candidates and rerank the positives over the

negatives. To operationalize this framework, we further introduce Mistral4SelectEM, which fine-tunes the LLM-based embedding model with CMRL for selective EM.

3. Selective entity matching

3.1. Task formulation

Selective EM involves identifying which candidates in a retrieved subset match a given query entity. Let $X = \{e_i\}_{i=1}^N$ represent a source database consisting of N query entities e_i , and $X' = \{e'_j\}_{j=1}^M$ represent a target database consisting of M candidate entities e'_j . Each entity is characterized by attributes with key-value pairs, and is typically structured in form of relational tables. Notably, there is no strict distinction between the source database X and the target database X' . The relationship between them is symmetric, meaning either can serve as the query set or candidate set depending on the context.

After an initial blocking step retrieves a subset $X'' = \{e'_j\}_{j=1}^K \in X'$, which is assumed to contain the K most semantically similar candidates for a given query entity, the selective EM focuses on identifying matches from this subset. To be more specific, the selective EM is to find an optimal language model as the selector LM_s that identify the true positive matches between a query entity and candidates in the subset X'' . The model outputs a binary match vector $y_i \in \{0, 1\}^K$, where $y_{ij} = 1$ indicates that candidate e'_j matches the query entity e_i , and $y_{ij} = 0$ otherwise. It maps the query entity and its candidate subset to a vector of predictions. The task can be formally expressed as:

$$LM_s : \{(e_i, X'')\} \rightarrow y_i \quad (1)$$

Depending on the size of K and the input length constraints of LM_s , the model may process the entire candidate subset X'' or only a part of it, supplemented by the query and an optional instruction prompt. If K exceeds the model's input length, a tournament strategy can partition X'' into smaller subsets to ensure complete coverage. For simplicity, it is assumed that the entire subset X'' can be processed in one pass.

3.2. Dataset and evaluation

Modern blocking methods can already retrieve and rank semantically similar candidates effectively, making it essential for EM methods to refine these blocking results further. However, the current datasets are insufficient to evaluate this refinement capability. To address this issue, we propose to recompile the existing datasets to include more challenging competitive HN samples for each query entity.

The recompilation process begins with the retention of labeled positive match pairs from the existing train, validation, and test sets. For each query entity in these positive pairs, we implement a blocking step by combining an embedding model with an efficient indexing and search algorithm. In this work, we adopt the SOTA LLM-based embedding model "Linq-Embed-Mistral" [34] and utilize the FAISS library [35] for indexing and searching. The embedding model encodes both query and candidate entities into vector representations, and the top- K candidates are retrieved and ranked based on semantic similarity. The choice of the embedding model and the selection of $K = 10$ is informed by the ablation study in Section 4.4.1. If the retrieved candidates include entities already present in the original dataset, their labels are directly transferred to the recompiled dataset. For all other retrieved candidates, negative labels are assigned. It ensures that each query entity is associated with exactly K candidates. In most cases, there is a single true positive candidate; however, there are instances where multiple true positive candidates exist.

The recompiled dataset is structured in two formats. The first format maintains pairwise data, consistent with the original dataset, for use in

pairwise EM methods. The second format adopts an IR-style representation, where the label for a query is a binary match vector $y_i \in \{0, 1\}^K$, indicating the position of true positive candidates in the top- K retrieved set.

This dataset and the evaluation aim to address two key research questions. RQ1: Can an EM method effectively distinguish true positive candidates from similar distractors? RQ2: How much can an EM method further improve the results delivered by a modern blocking method? To answer RQ1, we evaluate changes in the typical EM metrics, such as F1 score and precision, for SOTA pairwise matchers when tested on the original and the recompiled benchmark datasets. To answer RQ2, we first rerank candidates according to the predicted labels, ensuring that predicted positive entities are reranked higher than negatives. The internal order among positive or negative candidates remains consistent with the ranking produced by the blocker. Next, MRRP is calculated, which extends the widely used MRR metric from IR to align with the specific objectives of the EM task. This metric evaluates the ranking quality while penalizing false positives and missed detections. Let r denote the reranked indices of candidates after prediction, and y_i represent the ground truth binary labels. The reciprocal rank (RR) is calculated as:

$$RR = \begin{cases} \frac{1}{\min(r_{true})} & \text{if at least one true positive exists,} \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

where $rank_{true}$ represent the ranks of true positive candidates in the predicted order r

To incorporate penalties, we define FP as the count of false positives and FN as the count of missed true positives. The penalty factor is given by:

$$Penalty = \frac{1}{1 + FP + FN} \quad (3)$$

The final MRRP score is computed by applying the penalty factor to the reciprocal rank:

$$MRRP = \frac{1}{N} \sum_{i=1}^N (RR_i \times Penalty_i) \quad (4)$$

where N refers to the total number of query entities in the test set.

The MRRP resulting from the EM method is compared against the baseline MRRP obtained from the pre-trained embedding model. By analyzing the relative improvement in MRRP, we assess how effectively the EM method enhances the results delivered by the modern blocking method.

3.3. Modeling and fine-tuning

The left part of Fig. 3 illustrates the proposed end-to-end selective EM approach. The blocking step uses an LLM-based embedding model to retrieve candidate entities. This same embedding model is further fine-tuned for selective EM with LoRA [36], which updates only the LoRA weights while keeping the original embedding model weights unchanged. At the inference stage of Mistral4SelectEM, the original weights of the embedding model are merged with the fine-tuned LoRA weights. This ensures that the end-to-end EM process operates with a single set of LLM weights.

The right part of Fig. 3 details the fine-tuning strategy for Mistral4SelectEM. For modeling the selector LM_s , the embedding model is structured in a Siamese network as the backbone model, where its two instances share identical weights. The attributes of the query entity e_i and the K candidate entities $\{e'_j\}_{j=1}^K$ are concatenated to form a query string Q and K candidate strings $\{C_j\}_{j=1}^K$. These are separately fed into the Siamese network, and the last token embeddings are used to

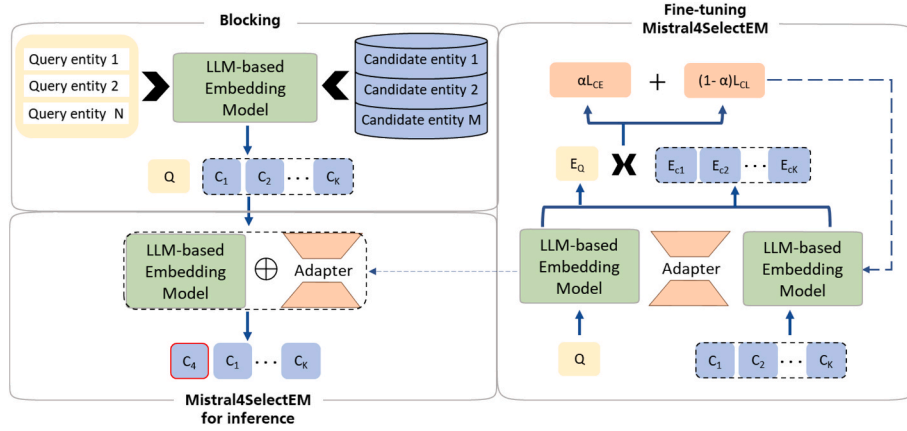


Fig. 3. Illustration of the end-to-end selective EM (left) and the fine-tuning strategy for Mistral4SelectEM (right). The entire process involves a single set of LLM weights, which is presented as the green block. The “red” adapter is the fine-tuned LoRA weights, which are merged with the embedding model in the inference stage for selective EM.

represent their respective embeddings:

$$E_Q = LM(Q), E_{C_j} = LM(C_j) \quad (5)$$

These embeddings are normalized using L_2 normalization:

$$E_Q \leftarrow \frac{E_Q}{\|E_Q\|_2}, E_{C_j} \leftarrow \frac{E_{C_j}}{\|E_{C_j}\|_2} \quad (6)$$

The similarity between the query embedding E_Q and each candidate embedding E_{C_j} is computed using matrix multiplication:

$$s_j = E_Q^T E_{C_j}, S = [s_1, s_2, \dots, s_j] \quad (7)$$

where $S \in \mathbb{R}^{1 \times K}$ is the similarity matrix.

For fine-tuning the selector LM_s , the similarity matrix S is used to compute two types of loss. The first is the cross-entropy loss $\mathcal{L}_{CE}$, which evaluates the softmax-normalized similarity scores:

$$p_j = \frac{\exp(s_j)}{\sum_{j=1}^K \exp(s_j)}, \mathcal{L}_{CE} = -\sum_{j=1}^K y_j \log(p_j) \quad (8)$$

where $y_j \in \{0, 1\}$ is the ground truth label. $\mathcal{L}_{CE}$ is designed to enhance the model's classification ability by putting high enough probability on the true positive candidate. This aligns with the standard setting of fine-tuning embedding models from base LLMs, since $\mathcal{L}_{CE}$ is a mathematically similar to the standard contrastive learning loss InfoNCE. In our observation, this improves training stability and accelerates convergence.

The second loss is the CMRL $\mathcal{L}_{CL}$, designed to separate positive and negative similarity scores. The similarity scores are divided into positive scores s_{pos} (for true positive pairs) and negative scores s_{neg} (for false pairs) according to the ground truth labels. The top- H highest negative similarity scores, representing HN, are selected for loss computation. For each HN, the difference between the negative score and the positive score is computed with a margin added to encourage separation:

$$diff_h = s_{neg} - s_{pos} + margin \quad (9)$$

where $diff_h$ is calculated for all HNs and broadcasted across all positives. Additionally, a softmax function is applied to weigh the contribution of each HN to the overall loss:

$$softmax_weight_h = \frac{\exp(diff_h)}{\sum_{h=1}^H \exp(diff_h)} \quad (10)$$

The Relu function is applied to ensure that only violations of the margin contribute to the loss. When $diff_h < 0$, it indicates that the positive is

already ranked higher than the HNs by at least the defined margin. Hence, no penalty is needed. The $\mathcal{L}_{CL}$ loss is defined as:

$$\mathcal{L}_{CL} = \sum_{h=1}^H softmax_weight_h \bullet Relu(diff_h) \quad (11)$$

The combined loss function used for backpropagation is a weighted sum of the two losses:

$$L = (1 - \alpha) \mathcal{L}_{CE} + \alpha \mathcal{L}_{CL} \quad (12)$$

where $\alpha \in [0, 1]$ is a weighting hyperparameter.

This design of actively selecting the top- H HNs ensures that backpropagation focuses on the most informative samples. This is especially beneficial in low-resource EM tasks where training datasets are typically much smaller than IR datasets, as presented in Table 2. For a given query entity, not all of the top-10 retrieved candidates from the blocker are semantically similar. This is due to the limited number of HNs in the target databases. Excluding trivial negatives, it avoids introducing “noise” into optimization and reduces computational waste. The introduction of a margin serves to enforce a “safe distance” between positives and HNs, further encouraging the model to rank true positives with greater confidence above similar distractors.

During fine-tuning, only the LoRA weights in the LLM are updated, while the LLM's main weights remain frozen. In the inference stage, the similarity matrix S is used to compute the probabilities $p_i = [p_{i,1}, p_{i,2}, \dots, p_{i,K}]$ using the softmax function. The binary prediction vector is obtained by applying an argmax operation over the probabilities to predict the most similar candidate entity e'_{pred} for the given query e_i .

4. Experiments

The first set of experiments aims to address RQ1 by evaluating pairwise EM methods on the recomputed datasets, which include more competitive HN samples. The results are presented and discussed in Section 4.2.2. The second set of experiments aims to address RQ2 by evaluating the effectiveness of the proposed Mistral4SelectEM and comparing its performance with SOTA approaches. The results are presented and discussed in Section 4.3.2. Section 4.4 presents the ablation study of Mistral4SelectEM, analyzing its components and the selection of hyperparameters.

4.1. Benchmark datasets

We employed four datasets in the experiments. The first three are widely used public benchmark datasets in the e-commerce domain [5,9–16], originating from the ER Benchmark datasets [37] and the

Magellan data repository [38]. These datasets are well-known in the EM community, as they are considered more challenging and have a relatively higher number of samples, especially positive samples, which are suitable for fine-tuning LLMs. Additionally, they are considered more challenging for existing EM methods [9–16].

The fourth dataset is a proprietary dataset constructed using a commercially licensed product classification repository, ECLASS [39]. ECLASS is an industrial-standard terminology repository that follows the IEC 61360 schema to describe professional terminologies across various sectors in the form of an ontology. Each entity in ECLASS is represented by several standardized attributes, among which “preferred name” and “definition” are the most relevant for EM tasks. In this dataset, ECLASS Version 12 Advanced serves as the target database, comprising 27,423 unique entities. The query database contains 2,522 entities derived from the manufacturing domain, describing equipment and processes. These query entities are organized in an ontology-like information model called the Asset Administration Shell (AAS), published by the Industrial Digital Twin Association (IDTA) [40]. Table 1 presents three representative matched pairs from this dataset. The dataset is characterized by text-heavy entities with detailed textual descriptions. The AAS entities may either be incomplete, merely containing brief entity names such as Atex2Gas, or very domain-specific descriptions. For instance, the AAS entity “MaxPlasticisingFlowRate” refers to a specific flow rate for injection molding machines, while the corresponding ECLASS entity provides a more general definition for the maximum flow rate. This disparity creates a more challenging scenario, as the AAS and ECLASS entities may not be semantically consistent. In practice, ECLASS entities serve as a standardized dictionary for AAS entities, which are used to describe highly specific domain items.

An overview of the dataset statistics is provided in Table 2. Four original datasets were recompiled using the process described in Section 3.2, resulting in a total of eight datasets. Notably, all originally positive-labeled pairs were retained in the recompiled datasets. The query entities from these positive pairs were then used to sample competitive HN candidates. Consequently, the negative samples differ in the recompiled datasets. In the recompiled datasets, samples in the “rAE” subset contain only one positive candidate per query, while other recompiled subsets may contain multiple positive candidates for some queries.

Table 1
Representatives of query entities and target entities from the AAS-ECLASS dataset.

Query entity (AAS)		Target entity (ECLASS)	
idShort	description	preferred name	definition
Atex2Gas	None	approval according to ATEX II / gas present	Information on whether a part is tested and approved according to ATEX II / gas or not
MaxPlasticisingFlowRate	The volume of material the machine can transfer into the mold in a second at maximum injection speed	max. flow rate	largest possible volume that can be moved per unit time through the section of the device
Representation	Geometric representation of the CAD model. ValueList: SolidBody, WireFrame, Surface, Mesh, PointCloud	representation	Graphical appearance

Table 2
Overview of dataset statistics.

Datasets	Domain	Samples	Pos.	Neg.
Abt-Buy (AB)	E-comm	9575	1028	8547
Walmart-Amazon (WA)	E-comm	10,242	962	9280
Amazon-Google (AG)	Software	11,460	1167	10,293
AAS-ECLASS (AE)	Industry	5600	800	4800
Recompiled AB (rAB)	E-comm	1023	1028	9202
Recompiled WA (rWA)	E-comm	917	962	8208
Recompiled AG (rAG)	Software	1081	1167	9643
Recompiled AE (rAE)	Industry	800	800	7200

4.2. Experiments for pairwise entity matching

4.2.1. Experimental setup and implementation

The first experiments are designed to evaluate the performance of SOTA pairwise EM methods on both the original and recompiled datasets. Several representative SOTA pairwise matchers are implemented, trained, and tested on the original datasets using the same train-validation-test splits provided in [9]. Their F1 scores on the original datasets are reported and compared to the results from the related work, ensuring that the implemented models perform as expected and are comparable with those in related works. This step validates the model implementations.

To address RQ1, the implemented matchers are evaluated on the recompiled datasets. This evaluation provides insights into how well pairwise EM methods can distinguish true positives from semantically similar distractors. The pairwise matchers implemented in the experiments are selected to represent three major categories of SOTA methods: fine-tuned PLMs, LLMs with prompt engineering, and fine-tuned LLMs. For fine-tuned PLMs, RoBERTa-large is chosen as the representative model. GPT4 with ICL is used to represent LLMs with prompt engineering. For fine-tuned LLMs, two variations are implemented: LLaMA2-13B-Chat-Gen as a fine-tuned generation model and LLaMA2-13B-Encoding as a fine-tuned encoding model [5]. The base models are sourced from the Huggingface model hub, with RoBERTa-large loaded in full precision and the LLaMA2 models utilizing 4-bit precision for memory-efficient base model loading.

The hyperparameters are carefully configured for each model. All experiments employed a batch size of 32. RoBERTa-large is fine-tuned using a learning rate of 5×10^{-5} , while the LLaMA2 model is optimized using a learning rate of 2×10^{-4} . For the LLaMA2 models, LoRA is employed to reduce fine-tuning costs. The LoRA rank is set to 512 for AB, AG, and WA and 128 for AE, depending on the dataset size. Not all the negative samples are involved in the training stage. We set the imbalance ratio to 3. The LoRA scaling factor α/r , which adjusts the effective learning rate, is set to 1. All models are fine-tuned for 10 epochs on an Nvidia-DGX GPU cluster with $8 \times$ A100-40G GPUs. To implement GPT4 with ICL, the model “gpt-4o-2024-05-13” is utilized via the REST API provided by Azure OpenAI. The prompt instruction for GPT-4 is consistent with that used for LLaMA2-13B-Chat-Gen, which reads: “You will read two sentence-like entities to be matched. Each entity has several attributes, such as name and description. Your task is to decide whether the two entities are matched (i.e., they refer to the same entity). Only answer ‘yes’ or ‘no’.” To enable ICL, the prompt includes additional exemplary pairs: two match pairs and two non-match pairs. These examples serve as reference cases to guide the model in its decision-making process.

The competitive pairwise matchers include the following SOTA methods: DeepMatcher [9], which represents training deep learning models from scratch for EM; Ditto [10], RoBERTa [10], and JointMatcher [12], which represent fine-tuned PLMs; COMEM [8] and APrompt4EM [14], which leverage prompt engineering with LLMs; and LLaMA3-8B [15] and GPT-4o-m [15], which are fine-tuned generative LLMs. It is important to note that many of these SOTA methods incorporate model-agnostic optimization strategies, such as reformulating entity attributes into natural language sentences and applying data

augmentation techniques to improve performance. In our implementation, we exclude these additional strategies to ensure a direct comparison of the performance across the three method groups.

4.2.2. Results and discussions

The results in Table 3 provide valuable insights into the performance changes of pairwise matchers when HN samples are introduced in the recompiled datasets. First, focusing on the right-side columns, our implemented RoBERTa-large, GPT4-ICL, LLaMA2-13B-Gen, and LLaMA2-Chat-13B-Encoding achieved SOTA F1 scores on the original datasets AB, WA, and AG comparable to those reported in related works. Notably, the fine-tuned models LLaMA2-13B-Gen and LLaMA2-Chat-13B-Encoding outperformed other methods on these datasets, demonstrating the effectiveness of the implemented pairwise matchers.

However, when evaluating the models on the recompiled datasets (shown in the left-side columns), all methods exhibited more or less F1 drops. To further examine this degradation, Table 4 highlights the percentage drop in F1 score and precision, as well as the percentage increase in false positive samples for the four implemented pairwise matchers. In most cases, a substantial decrease in precision and a sharp increase in false positive samples are observed across all models, clearly illustrating the impact of introducing HN samples in the recompiled datasets. Notably, LLaMA2-13B-Gen, which achieved the best performance on the original datasets, experienced the most severe performance degradation, with over a 160 % increase in false positive samples, on the dataset pair “WA $\rightarrow$ rWA”. In comparison, LLaMA2-Chat-13B-Encoding exhibited relatively more stable performance. The only exception to this trend is observed in the dataset pair “AB $\rightarrow$ rAB”, where all models achieved slightly better performance on the recompiled datasets. This anomaly is likely due to characteristics of the target database, which may not contain enough HN samples for query entities. Therefore, the recompiled “rAB” dataset includes easier-to-distinguish negative samples than the original AB dataset.

The results answer RQ1, “Can an EM method effectively distinguish true positive candidates from similar distractors?” that pairwise matchers, by design, evaluate candidate pairs independently and are thus vulnerable to falsely classifying semantically similar negatives. As a result, the precision of these models is significantly affected, with false positives increasing. This finding underscores the limitations of pairwise EM methods in handling HNs and highlights the need for a more robust matching strategy and evaluation framework in this realistic scenario.

4.3. Experiments for selective entity matching

4.3.1. Experimental setup and implementation

The second set of experiments is designed to evaluate the performance of the proposed Mistral4SelectEM on the recompiled datasets using the metrics MRRP and mean inference time per query entity (MITQ). These experiments serve two main purposes. First, the proposed method is compared against both the implemented pairwise matchers

and competitive rerankers to demonstrate its advantages in terms of both efficiency and effectiveness. Second, to address RQ2, the experiment assesses whether the proposed entity selector and the pairwise matchers can refine the initial results provided by the LLM-based blocker. This analysis evaluates the realistic contribution of each method in improving the performance of the initial blocking step.

Mistral4SelectEM is implemented as illustrated in Fig. 3. The method employs an LLM-based embedding model, Linq-Embed-Mistral, as the base model of the Siamese network. This embedding model was fine-tuned based on the Mistral-7B-Instruct model across various NLP tasks using large datasets [34]. For fine-tuning Mistral4SelectEM on EM datasets, the maximum sequence length is set to 128, and the batch size is set to 32, enabling the model to process 10 candidate entities alongside the query entity as a single input sample for contrastive learning with CMRL. The CMRL includes three hyperparameters: the weighting factor α , the margin, and the number of top hard negatives H . A grid search is performed to determine the optimal values: $\alpha = 0.6$, $\text{margin} = 0.5$, and $H = 3$. Since the model is instruction-tuned, the query entity is formatted together with an instruction prompt in the following pattern as input: “*Instruct: Given a query entity, retrieve a similar entity that matches the query.* \nQuery: {query entity}”.

To compare against competitive reranking-based methods, we adopt RankLLaMA [24] and RankGPT [28] to our task. RankLLaMA is fine-tuned on our datasets using the “repLLaMA-v1-7b-LoRA-passage” model provided by its authors with the standard contrastive loss. Similar to Linq-Embed-Mistral, this “repLLaMA-v1-7b-LoRA-passage” model is an embedding model fine-tuned based on LLaMA-2-7B. RankGPT, on the other hand, uses ZSP with the GPT-4 model (“gpt-4o-2024-05-13”) to directly select the most matched candidate entity from the retrieved subset. The prompt instruction for RankGPT reads: “*Given a query entity consisting of several attributes, select the most similar entity that semantically matches the query entity from the given candidates. Only return the index of the selected one without any explanations. For example, when the answer is index ‘3’, which refers to candidate4 which is the most matched candidate for the given query entity.*”.

For fine-tuning all LLMs, a learning rate of 2×10^{-4} is used. The LoRA rank is set to 64, with the LoRA scaling factor α/r set to 1. All models are fine-tuned for 10 epochs on the same hardware platform as the previous experiments.

4.3.2. Results and discussions

Table 5 presents the performance comparison of the proposed Mistral4SelectEM with SOTA pairwise matchers and rerankers on the recompiled datasets using MRRP and MITQ as evaluation metrics. The baseline MRRP is provided by the Linq-Embed-Mistral model, which retrieves the top 10 candidates ranked by their semantic similarity to the query entity. Although this SOTA embedding model achieves strong MRRP values, the proposed Mistral4SelectEM can significantly improve the baseline and achieve the best overall performance across all recompiled datasets.

Table 3

Evaluation of SOTA pairwise EM methods on the original benchmark datasets and the recompiled datasets in terms of F1 score in percentile. Bold highlights the best results.

rAB	rWA	rAG	rAE	Model	AB	WA	AG	AE
–	–	–	–	DeepMatcher [9]	62.8	66.9	69.3	–
–	–	–	–	Ditto [10]	89.3	86.8	75.6	–
–	–	–	–	RoBERTa [11]	90.9	85.9	70.2	–
–	–	–	–	JointMatcher [12]	–	87.2	75.7	–
85.9	74.5	65.3	61.3	RoBERTa-large	83.8	83.5	70.9	67.5
–	–	–	–	COMEM [8]	87.6	86.4	69.6	–
–	–	–	–	APrompt4EM [14]	–	80.2	67.1	–
94.1	78.2	66.6	67.2	GPT4-ICL	92.8	84.5	69.5	72.6
–	–	–	–	LLaMA3-8B [15]	87.3	65.65	50.0	–
–	–	–	–	GPT-4o-m [15]	94.1	78.8	80.2	–
96.3	85.8	76.8	72.6	LLaMA2-13B-Gen	95.8	90.7	81.0	78.4
93.8	84.6	73.4	71.8	LLaMA2-Chat-13B-Encoding	91.7	88.6	79.1	78.9

Table 4

Performance degradation for four implemented pairwise matchers when evaluated on the recompiled datasets. The table shows the percentage drop in F1 score (F1↓) and precision (P↓), along with the percentage increase in false positive rate (FP↑) for each model.

Model	AB → rAB			WA → rWA			AG → rAG			AE → rAE		
	F1↓	P↓	FP↑	F1↓	P↓	FP↑	F1↓	P↓	FP↑	F1↓	P↓	FP↑
RoBERTa-large	-2.1	-0.6	-18.0	9.0	16.0	128	5.6	6.5	40.7	6.2	8.9	86.3
GPT4-ICL	-1.3	-1.6	-12.5	6.3	5.4	66.7	2.9	4.3	30.5	5.4	6.8	27.4
LLaMA2-13B-Gen	-0.5	-0.9	-16.7	4.9	9.6	162	4.2	6.6	48.2	5.8	7.9	76.9
LLaMA2-Chat-13B-Encoding	-2.1	-4.2	-34.5	4.0	8.6	84.6	5.7	4.1	29.0	7.1	9.7	48.2

Table 5

Comparison of the proposed Mistral4SelectEM with SOTA pairwise matchers and rerankers on the recompiled datasets in terms of MRRP and MITQ. The first row shows the baseline delivered by the blocker using the Linq-Embed-Mistral model. The columns rAB-rAE represent the MRRP achieved by each method on the corresponding recompiled datasets, while the column MITQ reports the average inference time per query entity across all datasets.

Model	rAB	rWA	rAG	rAE	MITQ
Linq-Embed-Mistral	0.870	0.869	0.823	0.574	—
RoBERTa-large	0.842	0.738	0.594	0.472	7.5
GPT4-ICL	0.922	0.793	0.637	0.535	4.7
LLaMA2-13B-Gen	0.931	0.833	0.717	0.543	5.9
LLaMA2-Chat-13B-Encoding	0.927	0.814	0.705	0.569	1.3
RankLLaMA [24]	0.928	0.863	0.829	0.635	1.3
RankGPT [28]	0.733	0.655	0.682	0.406	1.6
Mistral4SelectEM	0.971	0.919	0.886	0.723	1.2

Among the competitive methods, the pairwise matchers (RoBERTa-large, GPT4-ICL, LLaMA2-13B-Gen, and LLaMA2-Chat-13B-Encoding) fail to improve the baseline MRRP on the rWA, rAG, and rAE datasets. By analyzing their predicted labels and MRRP for individual input samples, it is evident that these models suffer significant penalties due to relatively high false positive rates. This limitation aligns with the observations in Table 4, where pairwise matchers exhibited substantial performance drops in precision on recompiled datasets with the introduced HN. On the rAB dataset, pairwise matchers such as GPT4-ICL, LLaMA2-13B-Gen, and LLaMA2-Chat-13B-Encoding can improve the baseline MRRP. This performance also aligns with the results in Table 4, which shows these models achieving better F1 scores and precision on the recompiled rAB dataset than on the original AB dataset.

As a pointwise reranker, RankLLaMA outputs similarity scores for each query-candidate pair and uses these scores to rerank candidates. The top-1 candidate is then selected as the predicted positive. This method performs better than pairwise matchers due to fewer penalties from false or missing positives. Except on rWA, RankLLaMA consistently improves the baseline MRRP. In contrast, RankGPT struggles to handle the selection task effectively and performs poorly on all the datasets. Long and complex texts of concatenated entities overwhelm the ZSP approach.

In terms of MITQ, we observe that the encoding-based approach is significantly more efficient than the generation-based approach. For instance, LLaMA2-Chat-13B-Encoding achieves much faster inference times compared to LLaMA2-13B-Gen. This efficiency gap is primarily due to the computational overhead of the generation-based approach, where the generation head predicts each token by selecting the most likely option from the entire vocabulary dictionary. Surprisingly, RoBERTa-large is the slowest approach among the models. This unexpected result may be attributed to the implementation setup: The auto-device setting does not distribute the weights of smaller models like RoBERTa-large equally across multiple GPUs, unlike LLMs, which fully utilize GPU resources during inference. Additionally, LLMs leverage 4-bit precision for faster computation, while RoBERTa-large operates with full precision, which increases computational overhead and slows down the inference process.

In general, the selective EM approach offers greater efficiency,

particularly when compared to generative pairwise matchers such as GPT-4-ICL and LLaMA2-13B-Gen, as it processes the entire candidate set in a single forward pass. However, Mistral4SelectEM adopts a Siamese network structure, which introduces additional computational overhead. As a result, its efficiency is comparable to that of encoding-based pairwise matchers like LLaMA2-Chat-13B-Encoding rather than significantly surpassing them.

4.4. Ablation study

The ablation study aims to validate the contributions of key components in the proposed Mistral4SelectEM from two perspectives: the effect of model design choices, including the Siamese network structure and the selection of the embedding model, and the effect of the proposed CMRL and its associated hyperparameters.

4.4.1. Effect of the model design

The embedding model in Mistral4SelectEM serves two primary functions: retrieving the top-K candidate entities during the blocking step and acting as the backbone of the Siamese network for fine-tuning. An appropriate embedding model and a well-chosen value for K are critical. K should be small enough to make the subsequent entity selection more efficient and accurate while still ensuring that the true positive candidate is included and ranked highly within the top-K list. For the investigation, we compare several SOTA embedding models using standard metrics for entity alignment, i.e., MRR@ K and Hit Rate at K (Hits@ K), in order to identify the most suitable embedding model and an optimal K value for candidate selection.

The comparative embedding models include: the top-performing OpenAI model “text-embedding-3-large” [41]; the best-performing sentence-transformer-based model “all-mpnet-base-v2” [42]; and several leading open-source LLM-based embedding models from the Massive Text Embedding Benchmark (MTEB) leaderboard [43], such as “SFR-Embedding-Mistral” [44], “Linq-Embed-Mistral” [34], and “gte-Qwen2-7B-instruct” [45]. For each embedding model, we perform the blocking step using the FAISS library [35] for indexing and similarity search and evaluate performance using MRR@10, Hits@5, Hits@10, and Hits@20 across the four datasets.

Table 6 presents the evaluation results. These models were selected based on their strong results on the general-purpose MTEB leaderboard, and the results here confirm their generalization ability to more domain-specific EM tasks. Notably, all models, except for gte-Qwen2-7B-instruct, perform consistently well across datasets. First, we observe that LLM-based embedding models significantly outperform the conventional sentence-transformer model (all-mpnet-base-v2) across all metrics and datasets. Second, among the LLM-based models, both open-source SFR-Embedding-Mistral and Linq-Embed-Mistral slightly surpass the commercial text-embedding-3-large from OpenAI. When analyzing hit rates, we find that Hits@10 is nearly 100 % on three of the four datasets (rAB, rWA, rAG) for all top models. In the more challenging rAE dataset, Hits@10 drops to 0.87 with Linq-Embed-Mistral, outperforming SFR-Embedding-Mistral by a notable margin (0.87 vs. 0.82). Increasing K to 20 yields only marginal improvement (Hits@20 increases to 0.91), suggesting that additional retrieval depth does not add substantial value. Based on these findings, we select Linq-Embed-Mistral as the

Table 6

Comparing different embedding models for blocking in terms of MRR@10, Hits@5, Hits@10, and Hits@20.

Model	rAB				rWA				rAG				rAE			
	M@10	H@5	H@10	H@20	M@10	H@5	H@10	H@20	M@10	H@5	H@10	H@20	M@10	H@5	H@10	H@20
SFR-Embed-Mistral	0.90	0.99	1.0	1.0	0.86	0.97	1.0	1.0	0.84	0.97	0.99	1.0	0.65	0.77	0.82	0.91
Linq-Embed-Mistral	0.91	0.98	1.0	1.0	0.86	0.98	1.0	1.0	0.84	0.96	0.99	1.0	0.66	0.76	0.87	0.91
gte-Qwen2	0.20	0.15	0.17	0.18	0.30	0.29	0.34	0.37	0.34	0.36	0.39	0.42	0.19	0.13	0.15	0.19
text-embed-3-large	0.91	0.98	0.99	1.0	0.86	0.97	0.99	1.0	0.82	0.96	0.99	1.0	0.62	0.77	0.88	0.94
all_mpnnet-base-v2	0.76	0.87	0.96	0.99	0.70	0.85	0.93	0.97	0.68	0.81	0.89	0.93	0.49	0.58	0.66	0.75

embedding model of Mistral4SelectEM, along with $K = 10$ as the optimal retrieval size.

The proposed modeling and fine-tuning strategy, i.e., the Siamese structure and CMRL, is designed to be model-agnostic, aiming to improve the blocking results regardless of the underlying embedding model. To validate this aspect, we fine-tune Mistral4SelectEM using three different embedding backbones: SFR-Embedding-Mistral, Linq-Embed-Mistral, and all_mpnnet-base-v2. Fig. 4 presents the experimental results. The light-colored bars represent the baseline embedding models' MRR@10 performance, as previously shown in Table 6. Each model is then fine-tuned using the proposed selective EM approach. The corresponding dark-colored bars indicate the MRRP@10 performance of these fine-tuned models. The results demonstrate that Mistral4SelectEM consistently enhances retrieval performance across both advanced LLM-based embeddings and the lightweight sentence-transformer baseline.

In Mistral4SelectEM, the embedding model is structured in a Siamese network. In previous work, rerankers in IR are commonly fine-tuned using a cross-encoder structure [22–24], where the query and candidate entities are concatenated into a single input string with a special separator token. Our choice of the Siamese structure is motivated by the fact that LLM-based embedding models are originally fine-tuned using the contrastive loss within a Siamese setup. Preserving this architecture ensures greater stability and effectiveness when fine-tuning Mistral4SelectEM based on the pre-trained embedding model's weights. Fig. 5 presents the experimental comparison between these two modeling approaches against the baseline embedding model. The model fine-tuned using cross-encoder structure is denoted as Mistral4SelectEM-Cross. For the challenging datasets like rAG and rAE, the fine-tuned Mistral4SelectEM-Cross performs even worse than the baseline embedding model, while Mistral4SelectEM consistently overperforms the baseline model and Mistral4SelectEM-Cross.

4.4.2. Effect of the CMRL

The CMRL includes three hyperparameters: the weighting factor α , the margin, and the number of top hard negatives H . A grid search is

performed to determine the optimal values. As shown in Fig. 6, the best performance is achieved with $\alpha = 0.6$, margin = 0.5, and $H = 3$. Among these, the margin has relatively less influence on overall performance. It controls the degree of separation enforced between positive and negative candidates. A margin that is too large (approaching 1.0) can destabilize training and lead the model to overfit to HN. Conversely, a very small margin weakens the model's discriminative ability.

The weighting factor α controls the balance between $\mathcal{L}_{CE}$ and $\mathcal{L}_{CL}$. In the extreme case where $\alpha = 0$, only $\mathcal{L}_{CE}$ contributes to training. Fine-tuning with $\mathcal{L}_{CE}$ alone for selective EM more aligns with the standard setting of fine-tuning embedding models from base LLMs, since $\mathcal{L}_{CE}$ is a mathematically similar to the standard contrastive learning loss InfoNCE. In our observation, this improves training stability and accelerates convergence. InfoNCE is widely applied for training embedding models [34,44], and it is given by:

$$\mathcal{L}_{InfoNCE} = -\log\left(\frac{\exp(s_{pos}/\tau)}{\sum_{j=1}^K \exp(s_j/\tau)}\right) \quad (13)$$

where s_{pos} is the similarity of the true positive candidate, and s_j denotes the similarity of the negative candidate. τ denotes the temperature parameter that controls the sharpness of the distribution and the emphasis on HN. When $\tau = 1$, $\mathcal{L}_{InfoNCE}$ is mathematically equivalent to $\mathcal{L}_{CE}$. In this case, it loses the ability to emphasize HN. In practice, InfoNCE is often used with a small temperature value, such as 0.02 in [34], to amplify the contrast between similar and dissimilar pairs.

However, InfoNCE treats all negatives equally. This design is effective in large-scale datasets, where each query is associated with many relevant candidates, and the training set contains millions of examples. In contrast, EM datasets are significantly smaller, with far fewer training samples and limited relevant negatives per query entity. In this context, including all negative samples in the loss may dilute the learning signal from truly informative HNs. Taking this into account, we combine $\mathcal{L}_{CE}$ with $\mathcal{L}_{CL}$ in Mistral4SelectEM. $\mathcal{L}_{CE}$ ensures training stability and encourages the model to identify the true positive candidate without dis-

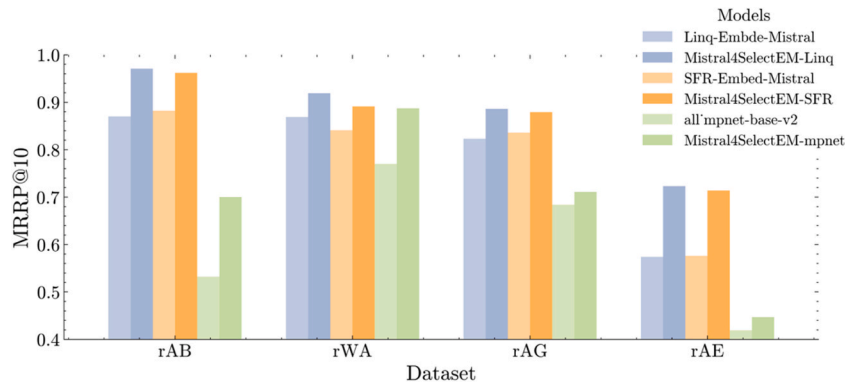


Fig. 4. Performance comparison of baseline embedding models (light color) and their fine-tuned counterparts (corresponding dark color) using the proposed Mistral4SelectEM.

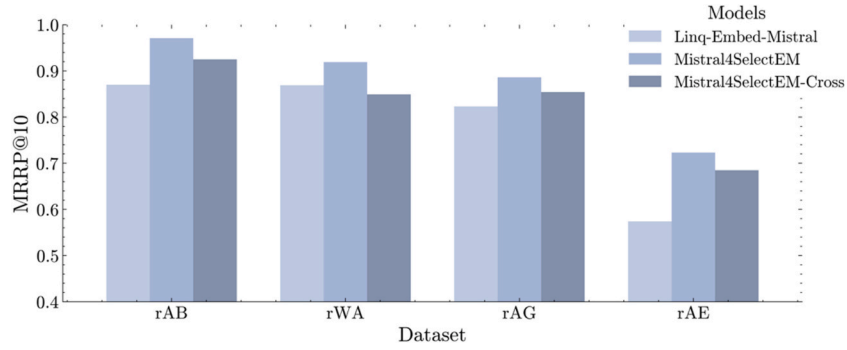


Fig. 5. Performance comparison of the baseline embedding model Linq-Embed-Mistral, the proposed Mistral4SelectEM, and Mistral4SelectEM-Cross fine-tuned using the cross-encoder structure.

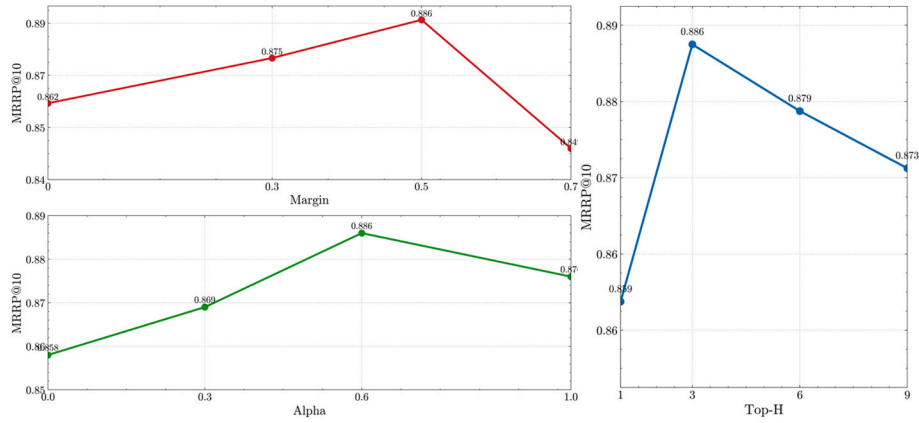


Fig. 6. Effect of three CMRL hyperparameter values on model performance in the rAG dataset.

tinguishing between easy and HNs. Meanwhile, $\mathcal{L}_{CL}$ explicitly focuses on a fixed number of HNs by selecting the top-H most similar negatives. The optimal weight factor α is found as 0.6, as shown in Fig. 6.

To further assess the design of CMRL, we compare it with both InfoNCE and Focal Loss. The latter also emphasizes hard examples by modulating the loss based on prediction confidence:

$$\mathcal{L}_{focal} = -(1 - s_{pos})^\gamma \log(s_{pos}) \quad (14)$$

where γ denotes the focusing parameter. When s_{pos} is high (i.e., easy examples), $(1 - s_{pos})^\gamma$ suppresses the loss.

Fig. 7 presents the comparison results. Both InfoNCE and Focal Loss are effective for fine-tuning LLMs in the selective EM setting and improve performance over the baseline embedding model. Our proposed Mistral4SelectEM, fine-tuned with the CMRL, outperforms that fine-

tuned with InfoNCE and Focal Loss. We observed that the model's predicted similarity scores for true positives and HNs are often very close in the early stages of fine-tuning. CMRL explicitly emphasizes HNs by selecting the top-H most challenging ones and applying a fixed margin to encourage clear separation from positives. This targeted mechanism appears to be more efficient than InfoNCE and Focal Loss for EM datasets, which contain fewer training samples, especially HN.

The number of top hard negatives H is used to exclude easy negatives from the $\mathcal{L}_{CL}$ loss computation, placing greater emphasis on the most challenging negative samples. These easier negatives are already accounted for in the $\mathcal{L}_{CE}$ loss. While traditional contrastive learning approaches often leverage a large pool of in-batch negatives to enhance representation learning, recent studies [34,44] have shown that selectively focusing on a smaller subset of HNs can lead to better performance. As shown in Fig. 6, setting $H = 3$ yields the best results in our

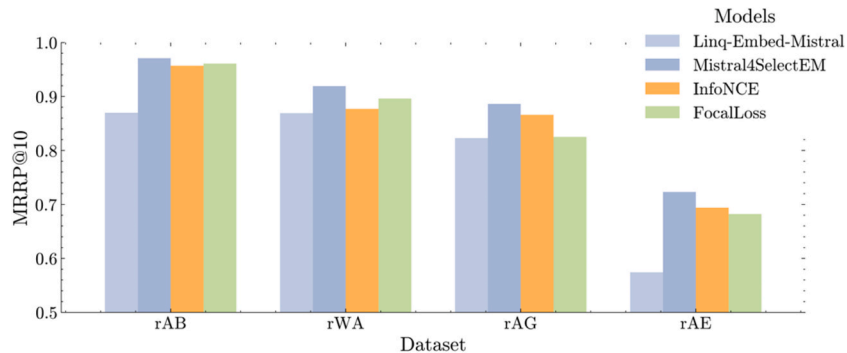


Fig. 7. Performance comparison of the baseline embedding model Linq-Embed-Mistral, the proposed Mistral4SelectEM, and the model fine-tuned using InfoNCE and Focal Loss.

case. However, although HNs contribute to more discriminative learning, overemphasizing them can potentially reduce the model's generalization ability across different datasets.

To address the concern regarding generalization, we conduct a cross-dataset evaluation by fine-tuning Mistral4SelectEM with different values of H on the rAG dataset and testing the resulting models on other datasets. The results presented in Fig. 8 reveal that setting $H = 1$ or 9 more impairs generalization ability, while the performance differences among $H = 3$ and 6 are relatively minor. Upon closer inspection of the EM datasets, we observe that for a given query entity, not all of the top-10 retrieved candidates from the blocker are semantically similar. This is due to the limited number of truly relevant or HN candidates in the target databases. For example, in the rAG dataset, the query entity “stuffit deluxe 11. aladdin systems. 49.99” yields only three semantically similar candidates: “allume stuffit deluxe 11 os x 10.4 or higher. nan. 79.99”, “allume stuffit deluxe 11 macintosh dvd pack sold only with a apple cpu. nan. 19.99”, and “allume stuffit deluxe 10.0 os x 10.4 or higher. smith-micro-software. 69.99”. The fourth candidate, “allume internet cleanup 4.0. smith-micro-software. 29.99”, is unrelated and regarded as an easy negative. In such cases, including more than a few HNs offers limited benefits. Setting $H = 3$ results in a performance degradation of approximately 2 % on rAG-rAB and rAG-rWA, yet Mistral4SelectEM still demonstrates strong generalization across standard EM benchmark datasets. A notable performance drop is observed only on rAG-rAE, which can be attributed to the rAE dataset being domain-specific and inherently different from the other benchmark datasets.

5. Conclusions

This paper addresses a critical gap in the field of EM, where standard pairwise EM methods struggle to handle semantically similar distractors within the candidate subset retrieved by modern blocking methods. By introducing more HN samples into the original benchmark datasets, this paper experimentally revisits the existing pairwise EM approach. Despite achieving SOTA performance on the original benchmark datasets, the best fine-tuned LLM-based pairwise matcher (i.e., LLaMA2-13B-Gen) exhibits an average F1 score drop of 3.6 %, a 5.8 % decrease in precision, and a 67.6 % increase in relative false positive rate on the recompiled datasets. This experimental finding emphasizes the inherent limitations of standard pairwise EM methods.

To address this gap, we introduce the selective EM framework, which redefines the task formulation, recompiles benchmark datasets, and proposes a new evaluation metric, MRRP. The proposed evaluation framework provides a more realistic and rigorous assessment by emphasizing the ability of EM methods to refine blocking results and reduce false positives. Using this framework, our evaluation reveals that even the best-performing fine-tuned LLM-based pairwise matcher can degrade performance. It shows an average 2.8 % drop in MRRP compared to the baseline blocker, namely the pre-trained embedding

model Linq-Embed-Mistral.

To address this gap, we propose a novel selective EM method, Mistral4SelectEM, which restructures a pre-trained embedding model into a Siamese architecture and fine-tunes it using the proposed CMRL. Instead of evaluating each candidate independently, Mistral4SelectEM compares the query entity against the entire candidate set in a single forward pass. This listwise selection approach leverages global context to better distinguish true positives from semantically similar distractors, significantly enhancing both inference efficiency and matching accuracy. Experimental results show that Mistral4SelectEM significantly improves retrieval performance. It boosts MRRP by an average of 9.6 % over the baseline embedding model, 12.4 % over the best-performing pairwise matcher, and 6.7 % over the best reranker. Additionally, it achieves the fastest inference time per query at 1.2 s. Ablation studies further validate the effectiveness of its key design components.

While selective EM shares similarities with listwise reranking approaches in IR, it addresses unique challenges specific to EM tasks. Unlike IR, where long texts in passage candidates pose the primary challenges, EM focuses on identifying the matched structured entities with shorter text descriptions. The results of this study suggest that selective EM approaches are more aligned with real-world EM scenarios and should be further explored and developed within the EM domain. In the current implementation, Mistral4SelectEM is limited to predicting a single positive candidate per query, as it relies on an argmax operation over the predicted similarity scores. However, in real-world scenarios, multiple true positives may exist. This limitation should be addressed in future work to enhance the model's applicability.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the author(s) used Grammarly and ChatGPT in order to improve language and readability. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the publication.

CRediT authorship contribution statement

Qian Ruan: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis. **Dachuan Shi:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Thomas Bauernhansl:** Supervision, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial

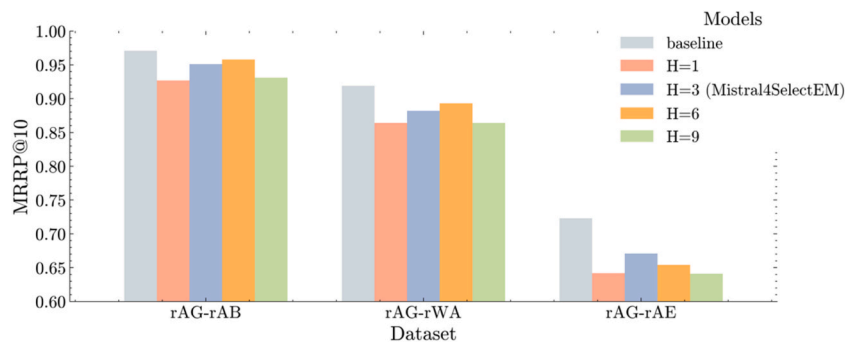


Fig. 8. Effect of H values on model generalization. Each group shows models fine-tuned on rAG and evaluated on rAB, rWA, or rAE, respectively. Baseline bars represent models both trained and tested on the same dataset. Each bar compares performance under different HN settings ($H = 1, 3, 6, 9$); $H = 3$ corresponds to the proposed Mistral4SelectEM configuration.

interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the Kopernikus project “Syn-ergie” (03SFK3A3-3; 03SFK3T2-3) and in part by the project H2Giga-FRHY (3HY112C). Both projects are funded by the German Federal Ministry of Education and Research.

Data availability

Data and code are publicly available at: <https://github.com/quickhdsdc/LLM4EntityMatching>

References

- [1] M.G. de Carvalho, A.H.F. Laender, M.A. Goncalves, A.S. Da Silva, A genetic programming approach to record deduplication, *IEEE Trans. Knowl. Data Eng.* 24 (2012) 399–412.
- [2] R. Estrada-Valenciano, V. Muñoz-Sánchez, H. De-la-Torre-Gutiérrez, An entity-matching system based on multimodal data for two major E-Commerce Stores in Mexico, *Mathematics* 10 (2022) 2564.
- [3] F. Brauer, M. Schramm, W. Barczynski, A. Loser, H. Do, Robust recognition of complex entities in text exploiting enterprise data and NLP-techniques, *Third International Conference on Digital Information Management*, 2008, pp. 551–558.
- [4] S. Kong, X. Huang, S. Li, G. Li, D. Zhang, Entity alignment method for aeronautical metrology domain based on multi-perspective entity embedding, *Adv. Eng. Inf.* 62 (2024) 102908.
- [5] D. Shi, O. Meyer, M. Oberle, T. Bauernhansl, Dual data mapping with fine-tuned large language models and asset administration shells toward interoperable knowledge representation, *Rob. Comput. Integr. Manuf.* 91 (2025) 102837.
- [6] H. Köpcke, E. Rahm, Frameworks for entity matching: a comparison, *Data Knowl. Eng.* 69 (2010) 197–210.
- [7] N. Barlaug, J.A. Gulla, Neural networks for entity matching: a survey, *ACM Trans. Knowl. Discov. Data* 15 (2021) 1–37.
- [8] T. Wang, X. Chen, H. Lin, X. Chen, X. Han, H. Wang, Z. Zeng, Le Sun, Match, Compare, or Select? An Investigation of Large Language Models for Entity Matching, 2024.
- [9] S. Mudgal, H. Li, T. Rekatsinas, A. Doan, Y. Park, G. Krishnan, R. Deep, E. Arcaute, V. Raghavendra, Deep Learning for Entity Matching, in: *Proceedings of the 2018 International Conference on Management of Data*, Houston TX USA, ACM, New York, NY, USA (2018) pp. 19–34. GitHub repository for benchmark datasets is [Online] Available: <https://github.com/anhaidgroup/deepmatcher/blob/master/Datasets.md#preprocessed-data>.
- [10] Y. Li, J. Li, Y. Suhara, A. Doan, W.-C. Tan, Effective entity matching with transformers, *VLDB J.* 32 (2023) 1215–1235.
- [11] U. Brunner, K. Stockinger, Entity Matching with Transformer Architectures - A Step Forward in Data Integration, *OpenProceedings.org*. (2020).
- [12] C. Ye, S. Jiang, H. Zhang, Y. Wu, J. Shi, H. Wang, G. Dai, JointMatcher: Numerically-aware entity matching using pre-trained language models with attention concentration, *Knowl.-Based Syst.* 251 (2022) 109033.
- [13] R. Peeters, C. Bizer, Entity Matching using Large Language Models (2023) [Online]. Available: [arXiv:2310.11244](https://arxiv.org/abs/2310.11244).
- [14] Y. Xia, J. Chen, X. Li, J. Gao, APrompt4EM: augmented prompt tuning for generalized entity matching (2024) [Online]. Available: <https://arxiv.org/abs/2405.04820v1>.
- [15] A. Steiner, R. Peeters, C. Bizer, Fine-tuning Large Language Models for Entity Matching. (2024) [Online]. Available: [arXiv:2409.08185](https://arxiv.org/abs/2409.08185).
- [16] D. Shi, P. Liedl, T. Bauernhansl, Interoperable information modelling leveraging asset administration shell and large language model for quality control toward zero defect manufacturing, *J. Manuf. Syst.* 77 (2024) 678–696.
- [17] H. Nie, X. Han, B. He, L. Sun, B. Chen, W. Zhang, S. Wu, H. Kong, Deep Sequence-to-Sequence Entity Matching for Heterogeneous Entity Resolution, in: *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, Beijing China, ACM, New York, NY, USA, 2019, pp. 629–638.
- [18] R. Zhang, B.D. Trisedya, M. Li, Y. Jiang, J. Qi, A benchmark and comprehensive survey on knowledge graph entity alignment via representation learning, *VLDB J.* 31 (2022) 1143–1168.
- [19] Bayu Distiawan Trisedya, Jianzhong Qi, Rui Zhang, Entity Alignment between Knowledge Graphs Using Attribute Embeddings, in: *Proceedings of the 2019 AAAI Conference on Artificial Intelligence*, 33(01), 297–304.
- [20] R. Zhang, Y. Su, B.D. Trisedya, X. Zhao, M. Yang, H. Cheng, J. Qi, AutoAlign: fully automatic and effective knowledge graph alignment enabled by Large language models, *IEEE Trans. Knowl. Data Eng.* 36 (2024) 2357–2371.
- [21] R. Wang, Y. Zhang, Pre-trained Language Models for Entity Blocking: A Generalization Study, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (2024) 8720–8730.
- [22] W.X. Zhao, J. Liu, R. Ren, J.-R. Wen, Dense Text Retrieval based on Pretrained Language Models: A Survey, 2022.
- [23] R. Nogueira, W. Yang, K. Cho, and J. Lin, “Multi-stage document ranking with BERT,” *CoRR*, vol. abs/1910.14424, 2019.
- [24] X. Ma, L. Wang, N. Yang, F. Wei, J. Lin, Fine-Tuning LLaMA for Multi-Stage Text Retrieval, 2023.
- [25] H. Zhuang, Z. Qin, K. Hui, J. Wu, Le Yan, X. Wang, M. Bendersky, Beyond Yes and No: Improving Zero-Shot LLM Rankers via Scoring Fine-Grained Relevance Labels, 2023.
- [26] Z. Qin, R. Jagerman, K. Hui, H. Zhuang, J. Wu, Le Yan, J. Shen, T. Liu, J. Liu, D. Metzler, X. Wang, M. Bendersky, Large Language Models are Effective Text Rankers with Pairwise Ranking Prompting, 2023.
- [27] Y. Zhu, H. Yuan, S. Wang, J. Liu, W. Liu, C. Deng, H. Chen, Z. Liu, Z. Dou, J.-R. Wen, Large Language Models for Information Retrieval: A Survey, 2023.
- [28] W. Sun, L. Yan, X. Ma, S. Wang, P. Ren, Z. Chen, D. Yin, Z. Ren, Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents, 2023.
- [29] R. Pradeep, S. Sharifmoghaddam, J. Lin, RankVicuna: Zero-Shot Listwise Document Reranking with Open-Source Large Language Models, 2023.
- [30] S. Chen, B.J. Gutiérrez, Y. Su, Attention in Large Language Models Yields Efficient Zero-Shot Re-Rankers, 2024.
- [31] R.G. Reddy, J. Doo, Y. Xu, M.A. Sultan, D. Swain, A. Sil, H. Ji, FIRST: Faster Improved Listwise Reranking with Single Token Decoding, 2024.
- [32] S. Yoon, E. Choi, J. Kim, H. Yun, Y. Kim, S. Hwang, ListT5: Listwise Reranking with Fusion-in-Decoder Improves Zero-shot Retrieval, 2024.
- [33] Q. Liu, B. Wang, N. Wang, J. Mao, Leveraging Passage Embeddings for Efficient Listwise Reranking with Large Language Models, 2024.
- [34] J. Kim et al., Linq-Embed-Mistral:Elevating Text Retrieval with Improved GPT Data Through Task-Specific Control and Quality Refinement, 2024 [Online]. Available: <https://huggingface.co/Linq-AI-Research/Linq-Embed-Mistral>.
- [35] M. Douze et al., The Faiss library (2024) [Online]. Available: <https://doi.org/10.48550/arXiv.2401.08281>.
- [36] E.J. Hu et al., LoRA: low-rank adaptation of large language models, (2021) [Online]. Available: <http://arxiv.org/pdf/2106.09685v2>.
- [37] H. Köpcke, A. Thor, E. Rahm, Evaluation of entity resolution approaches on real-world match problems, *PVLDB* 3 (1–2) (2010) 484–493.
- [38] P. Konda, et al., Magellan: toward building entity matching management systems, *PVLDB* 9 (12) (2016) 1197–1208.
- [39] ECLASS, An introduction to the standard. 2023. Available online: <https://eclass.eu/en/eclass-standard/introduction>.
- [40] IDTA, Registered AAS Submodel Templates. 2023. [Online]. Available: <https://industrialdigitaltwin.org/en/content-hub/submodels>.
- [41] OpenAI, Embeddings. 2024 [Online]. Available: <https://platform.openai.com/docs/guides/embeddings/embedding-models>.
- [42] huggingface, Model card: sentence-transformers/all-mpnet-base-v2. Available online: <https://huggingface.co/sentence-transformers/all-mpnet-base-v2>.
- [43] huggingface, Embedding Leaderboard MTEB(eng, v2).2025 [Online]. Available: <https://huggingface.co/spaces/mteb/leaderboard>.
- [44] R. Meng, et al., SFR-Embedding-Mistral: Enhance Text Retrieval with Transfer Learning. Salesforce AI Research Blog, 2024 [Online]. Available: <https://blog.salesforceairesearch.com/sfr-embedded-mistral>.
- [45] Z. Li et al., Towards general text embeddings with multi-stage contrastive learning, 2023, arXiv preprint arXiv:2308.03281.